Practical 1: Breadth First Search (BFS) and Iterative Depth First Search (DFS)

Aim

1. To implement the Breadth First Search algorithm to solve a given problem.
2. To implement the Iterative Depth First Search algorithm to solve the same problem.
3. To compare the performance and efficiency of both algorithms.

Theory

Breadth First Search (BFS): BFS explores nodes level by level. It uses a queue (FIFO — First In, First Out). BFS finds the shortest path in terms of the number of edges when all edges have equal cost.

Iterative Depth First Search (DFS): DFS explores one branch as deeply as possible before returning to another branch. Iterative DFS uses a stack (LIFO — Last In, First Out) instead of recursion.

Algorithm

BFS

1. Start from the initial node.
2. Insert the node into a queue.
3. Remove the first node from the queue.
4. If it is the goal node, return the path.
5. Otherwise, add its unvisited neighbours to the queue.
6. Repeat until the goal is found or the queue becomes empty.

Iterative DFS

1. Start from the initial node.
2. Push the node and its path onto a stack.
3. Pop the top node from the stack.
4. If it is the goal node, return the path.
5. Otherwise, push its unvisited neighbours onto the stack.

6. Repeat until the goal is found or the stack becomes empty.

Python Program

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': ['H'],
    'F': [],
    'G': [],
    'H': []
}

start = 'A'
goal = 'H'


# Breadth First Search
def bfs(graph, start, goal):
    queue = deque([(start, [start])])
    visited = set()
    nodes_visited = 0

    while queue:
        node, path = queue.popleft()

        if node in visited:
            continue

        visited.add(node)
        nodes_visited += 1

        if node == goal:
            return path, nodes_visited

        for neighbour in graph[node]:
            if neighbour not in visited:
```

```
            queue.append((neighbour, path + [neighbour]))

    return None, nodes_visited


# Iterative Depth First Search
def iterative_dfs(graph, start, goal):
    stack = [(start, [start])]
    visited = set()
    nodes_visited = 0

    while stack:
        node, path = stack.pop()

        if node in visited:
            continue

        visited.add(node)
        nodes_visited += 1

        if node == goal:
            return path, nodes_visited

        # Reverse order preserves the graph's left-to-right
        # neighbour order when popping from the stack.
        for neighbour in reversed(graph[node]):
            if neighbour not in visited:
                stack.append((neighbour, path + [neighbour]))

    return None, nodes_visited


bfs_path, bfs_count = bfs(graph, start, goal)
dfs_path, dfs_count = iterative_dfs(graph, start, goal)

print("BFS Path:", " -> ".join(bfs_path))
print("BFS Nodes Visited:", bfs_count)

print("\nIterative DFS Path:", " -> ".join(dfs_path))
print("Iterative DFS Nodes Visited:", dfs_count)
```

Output

BFS Path: A -> B -> E -> H
BFS Nodes Visited: 8

Iterative DFS Path: A -> B -> E -> H
Iterative DFS Nodes Visited: 5

Performance Comparison

Feature	BFS	Iterative DFS
Data structure	Queue	Stack
Search method	Level by level	Deep into a branch
Shortest path	Yes, for unweighted graphs	Not guaranteed
Memory usage	Usually higher	Often lower
Complete	Yes, for finite graphs	Yes, for finite graphs with visited-state handling
Time complexity	O(V+E)O(V+E)O(V+E)	O(V+E)O(V+E)O(V+E)

Here, V represents the number of vertices and E represents the number of edges.

Result

BFS and Iterative DFS were implemented on the same graph. Both algorithms found a path from the initial node A to the goal node H. BFS explores level by level, while DFS explores deeper branches first. The path and number of visited nodes depend on the graph and traversal order.

Practical 2: A* Search and Recursive Best-First Search (RBFS)

Aim

1. To implement the A* Search algorithm for a pathfinding problem.
2. To implement the Recursive Best-First Search algorithm for the same problem.
3. To compare the performance and effectiveness of both algorithms.

Theory

A Search:* A* selects the next node using the evaluation function:

f(n)=g(n)+h(n)

Where:

- g(n) = actual cost from the start node to node nnn.
- h(n) = estimated cost from node nnn to the goal.
- f(n) = estimated total path cost through node nnn.

Recursive Best-First Search (RBFS): RBFS uses heuristic values and recursively explores the most promising path. It maintains a limit based on the best alternative path and backtracks when the current path is no longer promising.

Algorithm

A Search*

1. Initialize the start node with cost zero.
2. Calculate f(n)=g(n)+h(n)f(n)=g(n)+h(n)f(n)=g(n)+h(n).
3. Select the node with the smallest evaluation value.
4. Update the costs of its neighbours.
5. Continue until the goal node is reached.

RBFS

1. Start from the initial node.
2. Calculate the estimated cost of each successor.

3. Select the successor with the lowest estimated cost.
4. Set a limit using the best alternative successor.
5. Recursively explore the selected successor.
6. Backtrack when its cost exceeds the current limit.
7. Stop when the goal is found.

Python Program

```
import heapq

graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 2)],
    'D': [('G', 2)],
    'E': [('G', 5)],
    'F': [('G', 2)],
    'G': []
}

# Heuristic estimates to goal G
h = {
    'A': 4,
    'B': 3,
    'C': 3,
    'D': 2,
    'E': 4,
    'F': 2,
    'G': 0
}


# A* Search
def astar(start, goal):
    heap = [(h[start], 0, start, [start])]
    best_g = {start: 0}
    expanded = 0

    while heap:
        f, g, node, path = heapq.heappop(heap)
```

```
        if g != best_g.get(node):
            continue

        expanded += 1

        if node == goal:
            return path, g, expanded

        for neighbour, cost in graph[node]:
            new_g = g + cost

            if new_g < best_g.get(neighbour, float('inf')):
                best_g[neighbour] = new_g
                new_f = new_g + h[neighbour]

                heapq.heappush(
                    heap,
                    (new_f, new_g, neighbour, path + [neighbour])
                )

    return None, float('inf'), expanded

# Recursive Best-First Search
def rbfs(node, goal, g, f_limit, path):
    if node == goal:
        return path, g, g

    successors = []

    for neighbour, cost in graph[node]:
        new_g = g + cost
        f_value = max(new_g + h[neighbour], g + h[node])

        successors.append([neighbour, new_g, f_value])

    if not successors:
        return None, float('inf'), float('inf')

    while True:
        successors.sort(key=lambda item: item[2])
```

```
        best = successors[0]

        if best[2] > f_limit:
            return None, float('inf'), best[2]

        if len(successors) > 1:
            alternative = successors[1][2]
        else:
            alternative = float('inf')

        result, cost, new_f = rbfs(
            best[0],
            goal,
            best[1],
            min(f_limit, alternative),
            path + [best[0]]
        )

        best[2] = new_f

        if result is not None:
            return result, cost, new_f


a_path, a_cost, a_expanded = astar('A', 'G')

r_path, r_cost, _ = rbfs(
    'A', 'G', 0, float('inf'), ['A']
)

print("A* Path:", " -> ".join(a_path))
print("A* Total Cost:", a_cost)
print("A* Nodes Expanded:", a_expanded)

print("\nRBFS Path:", " -> ".join(r_path))
print("RBFS Total Cost:", r_cost)
```

Output

A* Path: A -> B -> E -> G
A* Total Cost: 7
A* Nodes Expanded: 6

RBFS Path: A -> B -> E -> G
RBFS Total Cost: 7

The node-expansion count may vary if the graph or tie-breaking order is changed.

Performance Comparison

Feature	A* Search	RBFS
Search method	Best-first search	Recursive best-first search
Evaluation	g(n)+h(n)g(n)+h(n)g(n)+h(n)	g(n)+h(n)g(n)+h(n)g(n)+h(n), with an alternative-cost limit
Memory	Stores frontier nodes	Uses much less search memory
Backtracking	Not normally required in standard graph search	Used when the cost limit is exceeded
Optimality	With suitable admissible heuristics and correct graph handling	With suitable admissible heuristics and correct implementation
Main advantage	Efficiently retains promising alternatives	Memory efficient

Result

A* Search and RBFS were implemented on the same weighted graph. Both found the path A → B → E → G with total cost 7. A* stores frontier nodes, whereas RBFS reduces memory usage through recursive exploration and backtracking.

Practical 3: Decision Tree Learning

Aim

1. To implement the Decision Tree Learning algorithm to build a decision tree for a given dataset.
2. To evaluate the accuracy and effectiveness of the decision tree on test data.
3. To visualize and interpret the generated decision tree.

Theory

A Decision Tree is a supervised machine-learning algorithm used for classification and regression. It makes predictions by dividing data according to feature-based conditions.

The algorithm selects features that help separate the classes. The quality of a split can be measured using Gini impurity or entropy and information gain.

Accuracy is calculated as:

Accuracy=Correct PredictionsTotal Predictions×100\text{Accuracy}= \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times100Accuracy=Total PredictionsCorrect Predictions×100

Algorithm

1. Load the Iris dataset.
2. Divide the dataset into training and testing sets.
3. Create a Decision Tree Classifier.
4. Train the model using the training data.
5. Predict the classes of test data.
6. Calculate the accuracy.
7. Display the decision tree graph.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
```

```
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Create and train model
model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=3,
    random_state=42
)

model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Number of test samples:", len(y_test))
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))

# Visualize decision tree
plt.figure(figsize=(14, 8))

plot_tree(
    model,
    feature_names=iris.feature_names,
```

```
    class_names=iris.target_names,
    filled=True,
    rounded=True
)

plt.title("Decision Tree for Iris Classification")
plt.show()
```

Output

```
Number of test samples: 30
Accuracy: 96.67 %

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.91      1.00      0.95        10
   virginica       1.00      0.90      0.95        10

    accuracy                           0.97        30
```

The notebook also displays a graphical decision tree.

Interpretation

- The root node represents the first feature-based decision.
- Each branch represents a condition.
- Each leaf node represents a predicted class.
- The Iris dataset contains three classes: Setosa, Versicolor and Virginica.
- Accuracy measures how many test samples were classified correctly.

Result

A Decision Tree Classifier was trained using the Iris dataset. Its test accuracy was evaluated, the classification report was displayed, and the generated decision tree was visualized and interpreted.

Practical 4: Feed Forward Backpropagation Neural Network

Aim

1. To implement the Feed Forward Backpropagation algorithm to train a neural network.
2. To use a given dataset to train the neural network for a specific task.
3. To evaluate the performance of the trained network on test data.

Theory

A Feed Forward Neural Network consists of an input layer, one or more hidden layers and an output layer.

- Feed forward: Input data moves from the input layer towards the output layer.
- Backpropagation: The network calculates prediction errors and uses gradients to update weights.
- Training: Weights are updated repeatedly to reduce the loss function.
- Evaluation: The trained model is tested using unseen data.

In this practical, `MLPClassifier` from scikit-learn is used to implement a feed-forward neural network trained through backpropagation.

Algorithm

1. Load the Iris dataset.
2. Split the dataset into training and testing sets.
3. Standardize the input features.
4. Create a neural network with one hidden layer.
5. Train the model using backpropagation.
6. Predict the classes of test data.
7. Calculate the test accuracy.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Create neural network
model = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=(10,),
        activation='relu',
        solver='adam',
        max_iter=2000,
        random_state=42
    )
)

# Train model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Evaluate model
accuracy = accuracy_score(y_test, y_pred)

print("Training completed successfully.")
print("Number of test samples:", len(y_test))
```

```
print("Test Accuracy:", round(accuracy * 100, 2), "%")
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

Output

Training completed successfully.
Number of test samples: 30
Test Accuracy: Approximately 96.67 %

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.91	1.00	0.95	10
virginica	1.00	0.90	0.95	10
accuracy			0.97	30

Result

A feed-forward neural network was trained using backpropagation on the Iris dataset. The model predicted the flower classes and its performance was evaluated using test accuracy and a classification report.

Practical 5: Support Vector Machines (SVM)

Aim

1. To implement the SVM algorithm for binary classification.
2. To train an SVM model using a given dataset and optimize its parameters.
3. To evaluate the performance of the SVM model on test data and analyze the results.

Theory

Support Vector Machine is a supervised learning algorithm used for classification and regression.

SVM identifies a decision boundary called a hyperplane that separates classes. It attempts to maximize the margin between the classes.

Important parameters include:

- C: Controls the penalty for classification errors.
- Kernel: Determines how the decision boundary is formed.
- Gamma: Controls the influence of individual training samples for kernels such as RBF.

This practical uses the Breast Cancer dataset for binary classification.

Algorithm

1. Load the Breast Cancer dataset.
2. Divide the data into training and testing sets.
3. Standardize the features.
4. Create an SVM classifier.
5. Optimize the parameters using GridSearchCV.
6. Predict the test classes.
7. Calculate accuracy and display the classification report.

Python Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Create pipeline
pipeline = make_pipeline(
    StandardScaler(),
    SVC()
)

# Define parameter combinations
parameters = {
    'svc__C': [0.1, 1, 10],
    'svc__kernel': ['linear', 'rbf']
}

# Optimize parameters
grid = GridSearchCV(
    pipeline,
    parameters,
    cv=5,
    scoring='accuracy'
)

grid.fit(X_train, y_train)

# Predict test data
y_pred = grid.predict(X_test)
```

```
# Evaluate model
print("Best Parameters:", grid.best_params_)
print("Test Accuracy:", round(
    accuracy_score(y_test, y_pred) * 100, 2
), "%")

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=data.target_names
))
```

Output

Best Parameters: {'svc__C': 1, 'svc__kernel': 'linear'}
Test Accuracy: Approximately 98.25 %

Classification Report:

	precision	recall	f1-score	support
malignant	0.98	0.95	0.96	42
benign	0.98	1.00	0.99	72
accuracy			0.98	114

Analysis

- High accuracy indicates that the model correctly classified most test samples.
- Precision measures the correctness of positive predictions.
- Recall measures how many actual class members were identified.
- GridSearchCV selects parameters using cross-validation on the training data.

Result

An SVM binary classifier was trained on the Breast Cancer dataset. Its parameters were optimized using GridSearchCV, and its effectiveness was evaluated using test accuracy and the classification report.

Practical 6: AdaBoost Ensemble Learning

Aim

1. To implement the AdaBoost algorithm to create an ensemble of weak classifiers.
2. To train the ensemble model on a given dataset and evaluate its performance.
3. To compare the results with individual weak classifiers.

Theory

AdaBoost stands for Adaptive Boosting. It is an ensemble learning technique that combines multiple weak classifiers to create a stronger classifier.

A weak classifier performs relatively poorly on its own. AdaBoost gives more attention to incorrectly classified training samples in subsequent rounds and combines the predictions of the classifiers.

In this practical, a decision stump (a decision tree with depth 1) is compared with an AdaBoost ensemble using the Breast Cancer dataset.

Algorithm

1. Load the Breast Cancer dataset.
2. Divide the dataset into training and testing sets.
3. Train a single decision stump.
4. Predict the test classes using the decision stump.
5. Train an AdaBoost ensemble.
6. Predict the test classes using AdaBoost.
7. Compare the accuracy of both models.

Python Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score
```

```
# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
# Train a weak classifier
weak_model = DecisionTreeClassifier(
    max_depth=1,
    random_state=42
)
weak_model.fit(X_train, y_train)
weak_pred = weak_model.predict(X_test)
# Create AdaBoost model
# Support both newer and older scikit-learn versions
try:
    model = AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1),
        n_estimators=50,
        learning_rate=1.0,
        random_state=42
    )
except TypeError:
    model = AdaBoostClassifier(
        base_estimator=DecisionTreeClassifier(max_depth=1),
        n_estimators=50,
        learning_rate=1.0,
        random_state=42
    )

# Train AdaBoost
model.fit(X_train, y_train)

# Predict test data
boost_pred = model.predict(X_test)
```

```
# Compare accuracy
weak_accuracy = accuracy_score(y_test, weak_pred)
boost_accuracy = accuracy_score(y_test, boost_pred)

print("Weak Classifier Accuracy:",
    round(weak_accuracy * 100, 2), "%")

print("AdaBoost Accuracy:",
    round(boost_accuracy * 100, 2), "%")
```

Output

Weak Classifier Accuracy: 89.47 %
AdaBoost Accuracy: 95.61 %

These values are illustrative; the actual values are determined by the installed library and model configuration.

Performance Comparison

Feature	Decision Stump	AdaBoost
Number of classifiers	One	Multiple
Learning method	Single decision tree	Sequential ensemble
Error handling	Does not adapt to previous errors	Gives more attention to misclassified samples
Complexity	Simple	More complex
Performance	May be limited	Often improves predictive performance

Result

A weak decision-tree classifier and an AdaBoost ensemble were trained and evaluated. Their test accuracies were compared to demonstrate how ensemble learning can improve classification performance.

Practical 7: Naive Bayes Classifier

Aim

1. To implement the Naive Bayes algorithm for classification.
2. To train a Naive Bayes model using a given dataset and calculate class probabilities.
3. To evaluate the accuracy of the model on test data and analyze the results.

Theory

Naive Bayes is a supervised machine-learning algorithm based on Bayes' theorem. It assumes that features are conditionally independent given the class.

Bayes' theorem is:

P(C∣X)=P(X∣C)P(C)P(X)P(C\mid X)=\frac{P(X\mid C)P(C)}{P(X)}P(C∣X)=P(X)P(X∣C)P(C)

Where:

- P(C∣X)P(C\mid X)P(C∣X) = posterior probability of class CCC.
- P(X∣C)P(X\mid C)P(X∣C) = likelihood of the observed features.
- P(C)P(C)P(C) = prior probability of the class.
- P(X)P(X)P(X) = evidence.

`GaussianNB` is suitable for continuous numerical features. In this practical, it is used to classify Iris flowers and calculate the probability of each class.

Algorithm

1. Load the Iris dataset.
2. Split the dataset into training and testing sets.
3. Create a Gaussian Naive Bayes classifier.
4. Train the classifier.
5. Predict the test classes.
6. Calculate accuracy.
7. Display the predicted class and class probabilities for a test sample.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Create and train model
model = GaussianNB()
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Test Accuracy:", round(accuracy * 100, 2), "%")

# Display prediction for one sample
sample = X_test[0].reshape(1, -1)

predicted_class = model.predict(sample)[0]
probabilities = model.predict_proba(sample)[0]

print("\nActual Class:",
      iris.target_names[y_test[0]])
```

```
print("Predicted Class:",
      iris.target_names[predicted_class])

print("\nClass Probabilities:")

for name, probability in zip(
    iris.target_names, probabilities
):
    print(name, ":", round(probability, 4))

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

Output

The numerical probabilities depend on the selected test sample.

Test Accuracy: Approximately 93.33 %

Actual Class: setosa
Predicted Class: setosa
Class Probabilities:
setosa : approximately 1.0
versicolor : approximately 0.0
virginica : approximately 0.0

Analysis

- A higher predicted probability indicates greater model confidence in that class.
- The model predicts the class with the highest estimated probability.
- Accuracy measures the proportion of correctly classified test samples.
- Naive Bayes is simple and computationally efficient, but its independence assumption may not always hold.

Result

The Gaussian Naive Bayes classifier was trained on the Iris dataset. Class probabilities were calculated, and the model was evaluated using test accuracy and a classification report.

Practical 8: K-Nearest Neighbors (K-NN)

Aim

1. To implement the K-NN algorithm for classification or regression.
2. To apply the K-NN algorithm to a given dataset and predict the class or value for test data.
3. To evaluate the accuracy or error of predictions and analyze the results.

Theory

K-Nearest Neighbors is a supervised learning algorithm used for classification and regression.

It predicts a result by finding the KKK closest training samples to a new data point.

For classification, the most common class among the neighbours is selected. For regression, the values of the neighbours are typically averaged.

Euclidean distance is calculated as:

$$d(x, y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

In this practical, K-NN is used to classify Iris flowers with K=5

Algorithm

1. Load the Iris dataset.
2. Split the data into training and testing sets.
3. Standardize the features.
4. Create a K-NN classifier with K=5K=5K=5.
5. Train the classifier.
6. Predict the classes of the test samples.
7. Calculate accuracy and display the confusion matrix.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Create K-NN model
model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)

# Train model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Evaluate model
accuracy = accuracy_score(y_test, y_pred)

print("Value of K: 5")
print("Test Accuracy:", round(accuracy * 100, 2), "%")

print("\nFirst 5 Actual Labels:", y_test[:5])
```

```
print("First 5 Predicted Labels:", y_pred[:5])

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Output

```
Value of K: 5
Test Accuracy: Approximately 96.67 %

First 5 Actual Labels: [0 0 0 0 0]
First 5 Predicted Labels: [0 0 0 0 0]

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]
```

Analysis

The confusion matrix shows the number of correct and incorrect predictions for each class. The diagonal values represent correct predictions.

- K=1K=1K=1 considers only the nearest training sample and may be sensitive to noise.
- A very large KKK may oversimplify the decision boundary.
- Feature scaling is important because K-NN relies on distance calculations.

Result

The K-NN classifier was implemented using the Iris dataset. The model predicted flower classes, and its performance was evaluated using accuracy and a confusion matrix.

Practical 9: Association Rule Mining using Apriori

Aim

1. To implement Association Rule Mining using the Apriori algorithm to find frequent itemsets.
2. To generate association rules from frequent itemsets and calculate support and confidence.
3. To interpret and analyze the discovered association rules.

Theory

Association Rule Mining identifies relationships between items that frequently occur together in transactional data.

For example, customers who buy bread may also buy milk.

The Apriori algorithm identifies frequent itemsets by checking whether their support meets a minimum threshold.

Support

Support

$$\text{Support}(A \to B) = \frac{\text{Transactions containing } A \cup B}{\text{Total transactions}}$$

Confidence

$$\text{Confidence}(A \to B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

Lift

$$\text{Lift}(A \to B) = \frac{\text{Confidence}(A \to B)}{\text{Support}(B)}$$

A lift greater than 1 indicates a positive association in the dataset.

Algorithm

1. Create a transaction dataset.
2. Convert the transactions into a one-hot encoded table.
3. Apply the Apriori algorithm.
4. Identify frequent itemsets using minimum support.
5. Generate association rules using minimum confidence.
6. Display support, confidence and lift.
7. Interpret the discovered rules.

Python Program

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Transaction dataset
transactions = [
    ['Bread', 'Milk'],
    ['Bread', 'Diaper', 'Beer', 'Eggs'],
    ['Milk', 'Diaper', 'Beer', 'Cola'],
    ['Bread', 'Milk', 'Diaper', 'Beer'],
    ['Bread', 'Milk', 'Diaper', 'Cola'],
    ['Bread', 'Milk', 'Eggs'],
    ['Milk', 'Diaper', 'Beer'],
    ['Bread', 'Diaper', 'Beer']
]

# Convert transactions into a Boolean table
encoder = TransactionEncoder()

encoded_data = encoder.fit(
    transactions
).transform(transactions)

basket = pd.DataFrame(
    encoded_data,
    columns=encoder.columns_
```

```
)

print("Transaction Table:")
print(basket.astype(int))

# Find frequent itemsets
frequent_itemsets = apriori(
    basket,
    min_support=0.25,
    use_colnames=True
)

print("\nFrequent Itemsets:")
print(frequent_itemsets.to_string(index=False))

# Generate association rules
rules = association_rules(
    frequent_itemsets,
    metric='confidence',
    min_threshold=0.60
)

print("\nAssociation Rules:")

if rules.empty:
    print("No rules found for the selected thresholds.")
else:
    print(
        rules[
            ['antecedents', 'consequents',
             'support', 'confidence', 'lift']
        ].round(3).to_string(index=False)
    )
```

Output

The program prints the transaction table, frequent itemsets and the association rules. Representative results include:

Frequent Itemsets:

support	itemsets
0.750	(Bread)
0.750	(Milk)
0.750	(Diaper)
0.625	(Beer)

Association Rules:

antecedents	consequents	support	confidence	lift
(Beer)	(Diaper)	0.500	0.800	1.067

Other rules may also appear depending on the thresholds.

Analysis

- Support: The fraction of all transactions containing the item combination.
- Confidence: The fraction of transactions containing the antecedent that also contain the consequent.
- Lift: The strength of the association compared with the consequent's overall frequency.

For example, the rule Beer → Diaper has support 0.5 and confidence 0.8. This means 50% of transactions contain both items, and 80% of transactions containing Beer also contain Diaper.

An association rule indicates a relationship, not proof that one purchase causes another.

Result

The Apriori algorithm was implemented on a transaction dataset. Frequent itemsets and association rules were generated, and their support, confidence and lift were calculated and interpreted.

Practical 10: Demonstration of OpenAI / TensorFlow Tools

Aim

1. To explore and experiment with OpenAI or TensorFlow tools and libraries.
2. To perform a demonstration or mini-project showcasing the capabilities of the tools.
3. To discuss and present the findings and potential applications.

Theory

Artificial Intelligence tools and libraries help developers build intelligent applications.

TensorFlow is an open-source machine-learning framework used to create, train and evaluate neural networks.

OpenAI tools provide AI capabilities through available models and APIs. API-based demonstrations may require an API key and may incur charges depending on the service and account.

For this practical, TensorFlow is used to build a simple neural network that classifies Iris flowers. It can be run in Jupyter Notebook without an OpenAI API key.

Algorithm

1. Install and import TensorFlow and scikit-learn.
2. Load the Iris dataset.
3. Divide the data into training and testing sets.
4. Standardize the input features.
5. Create a neural network with one hidden layer and an output layer.
6. Compile the model using an optimizer and loss function.
7. Train the neural network.
8. Evaluate the model on test data.
9. Display the predicted classes and accuracy.

Python Program

```
import numpy as np
import tensorflow as tf

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Set random seed
tf.keras.utils.set_random_seed(42)

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Standardize features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create neural network
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(4,)),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])

# Compile model
model.compile(
    optimizer='adam',
```

```
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train model
model.fit(
    X_train,
    y_train,
    epochs=40,
    batch_size=8,
    verbose=0
)

# Evaluate model
loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)
# Predict the first five test samples
probabilities = model.predict(
    X_test[:5],
    verbose=0
)
predictions = np.argmax(
    probabilities,
    axis=1
)
print("Training completed successfully.")
print("Number of test samples:", len(y_test))
print("Test Loss:", round(float(loss), 4))
print("Test Accuracy:", round(float(accuracy) * 100, 2), "%")

print("\nFirst 5 Predicted Classes:")

for i, prediction in enumerate(predictions):
    print(
        "Sample", i + 1, ":",
        iris.target_names[prediction]
    )
```

Output

The exact values depend on the training process and software version.

Training completed successfully.
Number of test samples: 30
Test Loss: (calculated by the model)
Test Accuracy: (calculated by the model)

First 5 Predicted Classes:
Sample 1 : setosa
Sample 2 : setosa
Sample 3 : setosa
Sample 4 : setosa
Sample 5 : setosa

Findings and Applications

Findings

- TensorFlow can create and train neural networks.
- The neural network learns patterns from the training dataset.
- The trained model predicts the flower class for unseen samples.
- Test accuracy helps evaluate the model's performance.

Potential Applications

1. Image classification.
2. Speech recognition.
3. Medical image analysis.
4. Text classification.
5. Predictive analytics.
6. Object detection.

Result

TensorFlow was used to build, train and evaluate a neural network for Iris flower classification. The practical demonstrated the basic workflow of an AI tool and its potential applications in machine learning.

Viva Voce Questions and Answers

1. What is Artificial Intelligence?

Artificial Intelligence is a branch of computer science that develops systems capable of performing tasks that normally require human intelligence.

2. What is an intelligent agent?

An intelligent agent perceives its environment through sensors and performs actions through actuators to achieve a goal.

3. What is the difference between BFS and DFS?

BFS explores nodes level by level using a queue. DFS explores a branch deeply using a stack or recursion.

4. What is a heuristic function?

A heuristic function estimates the remaining cost from the current state to the goal state.

5. What is the difference between A* and RBFS?

A* maintains a frontier of promising nodes, while RBFS uses recursive exploration and a cost limit to reduce memory requirements.

6. What is a decision tree?

A decision tree is a supervised learning model that predicts an output by following feature-based decisions from a root node to a leaf node.

7. What is overfitting?

Overfitting occurs when a model learns the training data too closely and performs poorly on unseen data.

8. What is backpropagation?

Backpropagation calculates gradients of the loss function and helps update neural-network weights to reduce prediction errors.

9. What is a support vector?

A support vector is a training sample that helps determine the decision boundary of an SVM.

10. What is a weak learner?

A weak learner is a model that performs only moderately well on a prediction task and can be combined with other learners.

11. What is Bayes' theorem?

Bayes' theorem calculates the probability of a hypothesis using prior probability and observed evidence.

12. What is the role of K in K-NN?

K determines the number of nearest training samples used to make a prediction.

13. What is support in association rule mining?

Support is the proportion of transactions that contain a specified itemset.

14. What is confidence in association rule mining?

Confidence is the proportion of transactions containing the antecedent that also contain the consequent.

15. What is TensorFlow? TensorFlow is an open-source framework used to develop and train machine-learning models, including neural networks.

16. What is a training dataset? A training dataset is the data used to teach a machine-learning model.

17. What is a test dataset? A test dataset is used to evaluate a trained model on data that was not used to fit it.

18. What is a confusion matrix?

A confusion matrix compares actual and predicted classes to show correct and incorrect classifications.

19. What is classification accuracy? Classification accuracy is the percentage of test samples predicted correctly.

20. Why is feature scaling important?

Feature scaling brings numerical features to comparable scales. It is especially useful for distance-based algorithms such as K-NN and margin-based algorithms such as SVM.